Technical Assessment: Event Timeline Reconstruction Engine

Track: Python / Data Engineering / Problem Solving Internship
Level: Medium | **Suggested time:** 4-6 hours | **Language:** Python 3.10+
Data: Large, intentionally messy, fully synthetic e-commerce event logs. No customer data is included.

1. Context

An online store receives order events from independent systems (web, payment gateway, warehouse, courier, and returns). Event rows are not necessarily stored in the order they actually occurred. Some were replayed, some contain inconsistent information, and some are malformed.

Build a Python program that reconstructs a defensible order timeline and flags problems. The goal is **correctness, transparent reasoning, robust Python, and good testing**, not a dashboard, ML model, or a complicated architecture.

2. Files supplied

- `data/order_events.csv`: **main dataset (~181,000 rows / ~30,000 orders)**. Use this for development and final execution.
- `data/debug_subset.csv`: smaller subset for faster development; the main data is authoritative.
- `data/worked_example_input.csv`: short hand-checkable example.
- `data/worked_example_summary.csv`: exact expected summary for the short example.
- `data/worked_example_anomalies.csv`: exact expected anomaly report for the short example.
- `data/DATA_DICTIONARY.md`: details of all input fields and validation conventions.

3. Your task

Implement an executable CLI that:

1. Loads an arbitrary CSV with the specified input schema.
2. Normalizes and validates records according to the exact rules below.
3. Identifies duplicate event IDs and conflicting reuses of an event ID.
4. Reconstructs each order's timeline from **event_time** (not row order or received_at).
5. Applies the supplied business transition rules; ignores invalid state changes.
6. Exports one per-order summary and one anomaly report.
7. Handles missing/invalid fields without crashing or fabricating information.

4. Required command and deliverables

Your submission **must** support:

```
python main.py --input data/order_events.csv --output-dir output
```

It must create:

- `output/order_summary.csv`
- `output/anomalies.csv`

It must also work with another CSV passed via `--input` without code changes, retraining, or external services. CSV row ordering may differ between runs.

`order_summary.csv` - required columns

```
order_id,final_status,valid_events,anomaly_count,first_valid_event_time,last_valid_event_time
```

- **One row for each distinct nonempty order_id appearing anywhere in the input**, including records that are rejected.
- `final_status`: last accepted state, or `UNKNOWN` if no event was accepted.
- `valid_events`: number of accepted transitions (count `PLACED` as one).
- `anomaly_count`: number of rows in `anomalies.csv` attributed to this order.
- `first_valid_event_time` / `last_valid_event_time`: UTC `YYYY-MM-DDTHH:MM:SSZ`, or empty if no event was accepted.
- Sort output by `order_id` ascending. Numbers must be integers.

`anomalies.csv` - required columns

```
row_id,order_id,event_id,anomaly_type,details
```

- Include **one row for each rejected input row**, at most one anomaly per `row_id`.
- The `details` column can be any clear human-readable explanation.
- `order_id` and `event_id` should contain the normalized (trimmed) values; empty remains empty.
- Sort output by `row_id` ascending.
- **Allowed anomaly_type values, exactly:**

`DUPLICATE_EVENT`, `EVENT_ID_CONFLICT`, `MISSING_ORDER_ID`, `MISSING_EVENT_ID`, `MISSING_TIMESTAMP`, `INVALID_TIMESTAMP`, `UNKNOWN_EVENT`, `INVALID_TRANSITION`.

5. Input normalization and event identity

- Trim surrounding whitespace from every input value. Empty or whitespace-only cells are missing.
- Normalize `event_type` by stripping whitespace and converting to uppercase. **Do not** invent aliases or change underscores.
- Treat `event_id` and `order_id` as **case-sensitive** strings after trimming.
- `row_id` is unique and nonmissing. It represents a source ingestion-row identifier. **It is not event time.**
- For repeated nonempty `event_id`, choose the row with the **lexicographically smallest `row_id` as the canonical row**. This remains deterministic if the CSV is shuffled.
- For every other row with the same `event_id`, compare its `(order_id, normalized event_type, raw event_time after trimming)` tuple with the canonical row:
  - equal tuple -> `DUPLICATE_EVENT`;
  - different tuple -> `EVENT_ID_CONFLICT`.
- Both kinds of noncanonical rows are rejected before validation or timeline reconstruction.
- Any row with missing `event_id` is **not** grouped with other missing IDs.
- Other columns (`received_at`, `source_system`, `metadata`) are context only. They do not decide identity, chronological ordering, or transitions.

6. Validation precedence for canonical rows

Check the canonical row in this exact order, emitting **only the first applicable error**:

1. Empty `order_id` -> `MISSING_ORDER_ID`.
2. Empty `event_id` -> `MISSING_EVENT_ID`.
3. Empty `event_time` -> `MISSING_TIMESTAMP`.
4. Unparseable `event_time` -> `INVALID_TIMESTAMP`.
5. `event_type` is not in the supplied vocabulary -> `UNKNOWN_EVENT`.
6. If none of the above apply, the event is eligible for timeline reconstruction.

Supported event_time representations (all UTC):

- `2026-04-03T09:00:00Z`
- `2026-04-03 09:00:00`
- `2026-04-03T09:00:00+00:00`

Reject impossible dates/times. A timestamp without a timezone is **UTC**. Convert accepted timestamps to UTC for ordering and output. Seconds precision is sufficient.

7. Business state machine

The following are the **only** valid next statuses. A transition into an identical status is not valid unless listed (none are).

Current state	Allowed next event_type
`START` (before any valid event)	`PLACED`
`PLACED`	`PAID`, `CANCELLED`
`PAID`	`PACKED`, `CANCELLED`
`PACKED`	`SHIPPED`, `CANCELLED`
`SHIPPED`	`OUT_FOR_DELIVERY`
`OUT_FOR_DELIVERY`	`DELIVERED`
`DELIVERED`	`RETURN_REQUESTED`
`RETURN_REQUESTED`	`RETURNED`
`RETURNED`	`REFUNDED`
`CANCELLED`	none
`REFUNDED`	none

Chronological processing: group eligible rows by trimmed `order_id`; sort by (parsed `event_time`, trimmed `event_id`, `row_id`) ascending. Evaluate events sequentially from `START`.

If a proposed transition is not allowed, report `INVALID_TRANSITION`, do **not** update the current state, and keep evaluating subsequent events. Do not infer missing intermediate events. A valid prefix of a timeline still counts even if later events are invalid.

8. Anomaly accounting

Duplicate/conflict checks are performed **before** single-row validation, and single-row validation occurs **before** state-transition checks. Each rejected `row_id` appears once only.

A rejected row with a nonempty `order_id` is counted toward that order's `anomaly_count`, even if it never entered the state machine. Rows with empty `order_id` appear in `anomalies.csv` but do **not** contribute to any order's `anomaly_count`.

Every **nonempty** `order_id` appearing in any input row gets an order summary, including an order appearing only in a rejected conflict row. Use `UNKNOWN` if it has no accepted event.

9. Example and tricky cases

The worked example intentionally includes unordered rows, a duplicated event ID, an event-ID conflict, cancellation followed by delivery, invalid timestamps, missing IDs, and unknown event types.

Read its three files together. Your program's output on `worked_example_input.csv` should match the supplied two expected CSVs (ignoring the free-text details description).

Important: `received_at` can be much later than `event_time`. It is not the ordering key. Having an event with an earlier row position does not make it canonical if the event IDs differ; only conflicting rows sharing one `event_id` use the `row_id` canonical rule.

10. Technical constraints and acceptable approaches

- Python 3.10+, standard library or free open-source packages (e.g. pandas) are acceptable.
- No external network calls, private data, API keys, paid APIs, or ML model are needed.
- A command should finish on the full provided dataset on a typical laptop without manual modifications.
- Output must be deterministic across reruns and changes to input CSV row ordering.
- AI coding assistants are allowed. You must understand and explain your solution; disclose significant tools or packages used in the README.
- Do not use the worked example answers as a hardcoded lookup. Your logic must work with the large data and other input orders.

11. Submission checklist

Submit a **single ZIP or Git repository** with:

- `main.py` (plus any supporting Python modules), supporting the required CLI.
- `requirements.txt` only if you use external packages.
- A `README.md` containing setup, commands, design decisions, complexity considerations, and limitations.
- **At least 5 meaningful unit tests** (duplicates, time ordering, invalid transitions, missing/unknown values, deterministic ties/reproducibility).
- `output/order_summary.csv` and `output/anomalies.csv`, generated on the complete supplied main dataset.
- A short **analysis note (maximum 1 page)** with: total orders, event and anomaly counts, 2-3 interesting edge cases you found, and one engineering tradeoff.

Don't include the large original input CSV in your submission unless explicitly requested.